

Smart Medical Records Engine

Helps a doctor find the one thing in a patient file that matters, before the appointment ends.

Taiwo Alabi · MSc Artificial Intelligence, National College of Ireland

Contents

1. The problem
 2. What it does
 3. How it works
 4. The safety rail that matters most
 5. Who is allowed in
 6. The data model
 7. Why self-hosted
 8. Verified: thirty attempts to leak, zero leaks
 9. Honest limitations
 10. Running it
-

1. The problem

A doctor has ten minutes with you.

Somewhere in your file, which might be three years of notes from different visits, different doctors, different hospitals, there is a line saying you reacted badly to a particular medicine.

Your doctor is about to prescribe you something. Do they remember that line? Did they ever see it? Nobody reads three years of notes in ten minutes.

That is the whole problem. Not a modelling problem, a retrieval problem with a consequence attached.

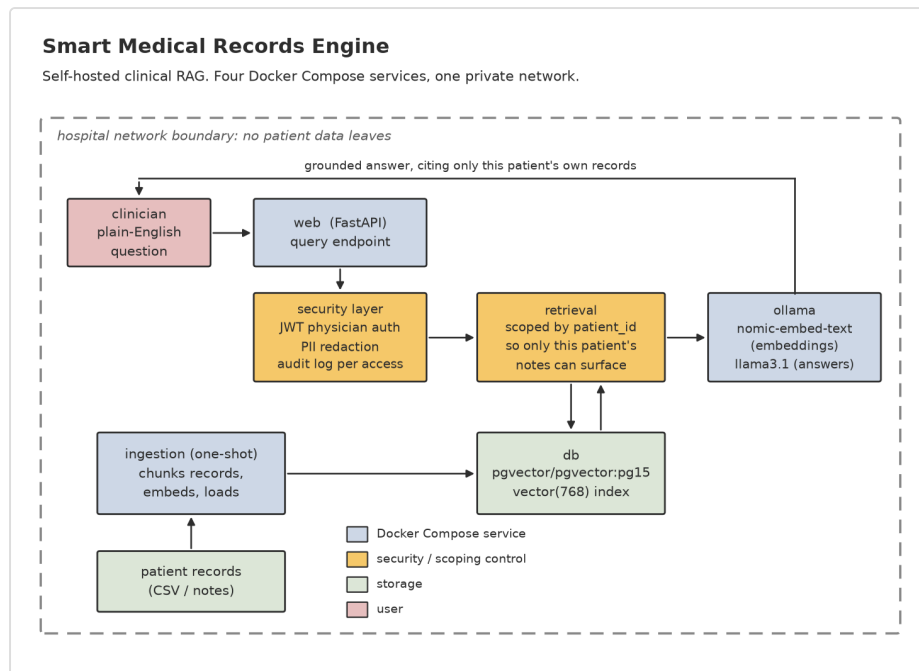
2. What it does

A search box a doctor can talk to normally. They ask *"has this patient ever reacted badly to any medication?"* and it answers from **that patient's own notes**, showing the actual note it found the answer in.

Three things make it trustworthy, and all three matter more than the clever part:

- **It only reads one file.** Not a similar patient's, not someone with the same illness. Verified: section 8.
 - **Records never leave the hospital.** The model runs on the hospital's own machine.
 - **Every access is written down.** If anyone asks who opened a patient's file, there is an answer.
-

3. How it works



Four Docker Compose services on one private network. Orange marks the controls that constrain what the system is allowed to do.

A question arrives at the FastAPI service. It passes the auth check, gets embedded by `nomic-embed-text`, and is used to search that patient's notes in Postgres with `pgvector`. The retrieved notes are handed to a locally-running `llama3.1` as context, and the answer comes back quoting them.

Ingestion runs once as a separate service: it chunks the records, embeds them, and loads them with their patient reference attached.

4. The safety rail that matters most

Retrieval is **scoped by patient before it ranks by similarity**.

This sounds like a detail. It is the whole design. A general vector search over all notes would happily return another patient's record if that record happened to be the closer semantic match, and it often would, because two people with the same condition produce very similar notes.

So the filter is not a ranking preference applied afterwards. Wrong-patient results are excluded from the candidate set entirely, and the constraint is carried in the schema rather than left to application code.

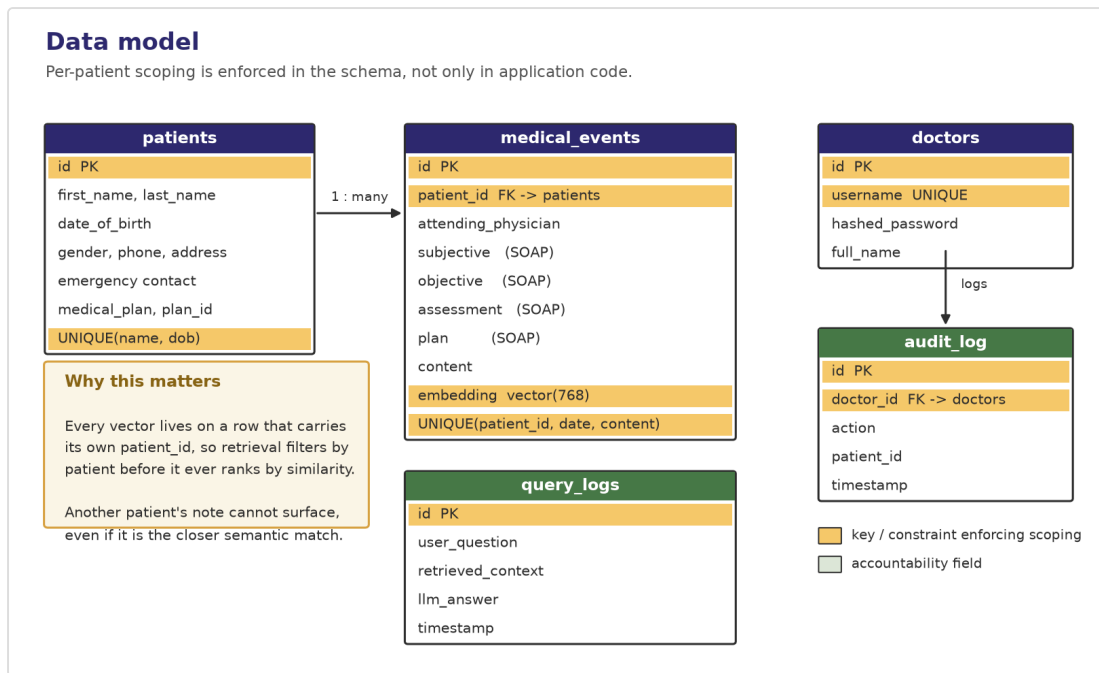
5. Who is allowed in

Physician authentication with JWT, passwords hashed rather than stored.

PII redaction runs before text reaches the model, so the language model never receives identifying details it does not need to answer the question.

Every access writes an `audit_log` row against the doctor who made it. A system handling medical records that cannot answer "who looked at this, and when" is not deployable regardless of how well it retrieves.

6. The data model



The `patient_id` foreign key on every embedded row is what enforces scoping at the database level.

Notes are stored in SOAP format, subjective / objective / assessment / plan, because that is how clinicians already write them. Matching the existing format matters more than choosing a neater one: it means records can be ingested as they are.

Each note carries a `vector(768)` embedding on the same row as its `patient_id`, so the scoping constraint and the searchable content cannot drift apart.

7. Why self-hosted

Every model in this system runs locally through Ollama. No API calls, no data sent out.

That decision costs accuracy. A hosted frontier model would answer better. But patient records leaving a hospital network is not a trade-off a hospital can make, so the ceiling on model quality is fixed by the constraint, not the other way round.

The whole stack, API, database and language model, runs across four Docker Compose services on one machine.

8. Verified: thirty attempts to leak, zero leaks

The claim in section 4 is now tested rather than asserted. I ran thirty adversarial probes against the live application, through the real path: signed in as a physician, asking questions designed to pull another patient's notes into the answer. Near-verbatim copies of a decoy patient's most distinctive lines. Semantic twins of the same clinical facts. Broad fishing questions. Queries naming the wrong patient outright.

Each probe carried a vacuity guard. Before a probe counts, I ran the same search without the patient filter, to confirm the unscoped search would have surfaced another patient's note first. All thirty passed that check: in 30/30 cases, the wrong patient's note would have been the top result. Scoped retrieval leaked 0/30.

What this is not: a penetration test. No auth bypass was attempted, no injection, no infrastructure attacks. It verifies the scoping design through the application's own query path, which is the design this system stands or falls on.

Reproduce:

```
python leak_probes/seed_fixture.py    # seeds a target patient + two near-  
duplicate decoys  
python leak_probes/run_probes.py      # runs the same thirty probes, prints a  
results table
```

9. Honest limitations

- **It has not been tested with real clinicians.** The retrieval works; whether the interface fits a ten-minute consultation is unproven, and that is the claim that matters.
- **The data is synthetic.** Real notes are messier: abbreviations, inconsistent formatting, scanned documents.
- **A local `llama3.1` is weaker than a hosted model.** Accepted deliberately, but it means answers on complex questions are less reliable than the retrieval underneath them.
- **No formal evaluation set.** Leakage was the priority, and it is measured in section 8: 0/30 across thirty adversarial probes. Answer quality against a labelled benchmark remains unmeasured.

10. Running it

```
docker compose up  
python scripts/create_doctor.py      # create a physician account
```

Ollama pulls `nomic-embed-text` and `llama3.1` on first run. Ingestion loads the records and builds the embeddings. Nothing reaches the internet after the model pull.

Stack: Python · FastAPI · PostgreSQL + pgvector · Ollama (llama3.1) · Docker Compose · JWT